Sri Lanka Institute of Information Technology



Final Evaluation - Group Report

2026-Y2-S1-MLB-B11G2-05

Optical Character Recognition - Classifying Letters from Shape Descriptors (UCI Letter Recognition Dataset)

IT25101436	Danishiyan G.J.
IT25102227	Thrishodaran V.
IT25103191	Almeida K.K.O.
IT25100513	Nirushiya A
IT25102618	Muhunthan R.
IT25100064	Wijayagunasekara L.A.V.D.S

IT2011 – Artificial Intelligence and Machine Learning

B.Sc. (Hons) in Information Technology

Year 2, Semester 1 – 2026

Contents

1. Introduction and Problem Statement
2. Dataset Description
3. Preprocessing & EDA
4. Model Design and Implementation
5. Evaluation and Comparison
6. Ethical Considerations and Bias Mitigation
7. AI Tool Usage Declaration and Transparency
8. Reflections and Lessons Learned
9. References

Introduction and Problem Statement

Optical Character Recognition (OCR) turns images of printed or written characters into machine-readable text. It underpins document digitisation, form processing, postal sorting and accessibility tools. A recognition system is only useful if it is accurate across all letters, because a single systematically misread letter corrupts names, codes and addresses.

This project frames OCR as a **supervised, 26-class classification problem**: given 16 numeric descriptors that summarise the shape of a scanned capital letter (bounding box, pixel counts, statistical moments and edge counts), predict which letter of the English alphabet (A–Z) it is. Because all 26 classes are almost equally frequent, accuracy is a meaningful headline measure, but macro-averaged precision, recall and F1 are reported as well so that no single letter can hide behind the others.

Beyond classification, the group asked three further questions: how much letter structure exists without labels (clustering), how much predictive information survives compression of the features (dimensionality reduction), and whether a neural network adds value over classical models (deep learning). The objectives were to:

- clean and prepare the assigned dataset through six preprocessing techniques, one owned and explained by each member (Progress Review I), chained into one shared pipeline;
- design, implement and tune six models across four families – classification, clustering, dimensionality reduction and deep learning – one per member;
- evaluate every model with suitable metrics and cross-validation, and compare the optimum result of each member on a common protocol;
- reflect critically on the ethics, limitations and process, including defects found and corrected along the way.

Dataset Description

The assigned dataset is the **UCI Letter Recognition Data Set**, donated by David J. Slate (Odesta Corporation, 1991) [1,2]. Each instance describes one capital letter rendered in one of 20 distorted commercial fonts; the images were converted by the dataset creators into 16 integer attributes scaled to the range 0–15.

Property	Value
Instances	20,000 raw; 18,668 after removing 1,332 exact duplicate rows (6.66%)
Original attributes (16)	x-box, y-box, width, high, onpix (position/size and pixel count); x-bar, y-bar, x2bar, y2bar, xybar, x2ybr, xy2br (statistical moments); x-ege, xegvy, y-ege, yegvx (edge counts)
Engineered attributes (5)	box_area, aspect_ratio, pixel_density, edge_ratio, total_variance – added in the pipeline, giving 21 predictors
Target	lettr (A–Z, 26 classes), label-encoded to lettr_encoded (0–25)
Missing values	None – verified in code with isnull().sum(), not only taken from the documentation
Class balance	Raw: 734–813 examples per letter. After de-duplication: 524–782 (max/min ratio ≈ 1.5), i.e. near-uniform but not perfectly balanced

Suitability. The dataset has a clear categorical label, a size (18,668 usable rows) that is comfortable for every algorithm used here, no missing data, and a well-defined real-world task. Its main quality issue is the duplicate rows, which would leak across a train/test split. Because the attributes are pre-extracted tabular descriptors rather than raw pixel images, every model treats the task as tabular classification; a convolutional network was deliberately not attempted, as it depends on spatial neighbourhoods that these aggregated descriptors no longer contain. The same property also limits how far results transfer to real scanned or handwritten text (Section 6).

Preprocessing & EDA

Each member implemented, explained and visualised one technique in Progress Review I. The six techniques were chained into a single pipeline (*group_pipeline.ipynb*), whose output *processed_letter_recognition.csv* (18,668 rows × 23 columns: 21 features, lettr and lettr_encoded) is read by every model.

Stage	Technique	Owner	Effect on the data
1	Duplicate removal	Nirushiya A.	1,332 exact duplicates removed; 20,000 → 18,668 rows
2	Outlier capping (IQR, k = 1.5)	Thrishodaran V.	10,209 extreme values clipped to the IQR bounds; no rows deleted
3	Feature engineering	Danishiyan G.J.	5 shape/density features added; 16 → 21 numeric columns
4	Target encoding	Wijayagunasekara L.A.V.D.S.	A–Z → 0–25 (LabelEncoder)
5	Standardisation	Almeida K.K.O.	StandardScaler on all 21 features (mean 0, std 1)
6	Feature ranking & PCA check	Muhunthan R.	ANOVA F-ranking and PCA analysis (21 → 13 components = 95.2% variance); data not altered

Pipeline order. Duplicates are removed first so that every later statistic is computed on distinct rows. Outliers are capped rather than deleted, because deletion could remove entire letter styles and unbalance the classes. Engineered features are created before encoding and scaling so that they are standardised together with the originals; scaling precedes PCA, which is scale-sensitive.

Data-quality correction. Three engineered features (aspect_ratio, pixel_density, edge_ratio) are ratios, and some rows have a zero denominator. The first implementation added a tiny constant before dividing, which merely turns a division by zero into a division by a near-zero number and produced values in the hundreds of thousands. These distorted the scaling and the principal components. Flooring each denominator at 1 bounds the ratios (maximum aspect_ratio 5.00, pixel_density 1.00, edge_ratio 7.00); PCA then needed 13 instead of 14 components for 95% variance, and edge_ratio became the top-ranked feature. All reported results use the corrected dataset.

Exploratory data analysis – findings

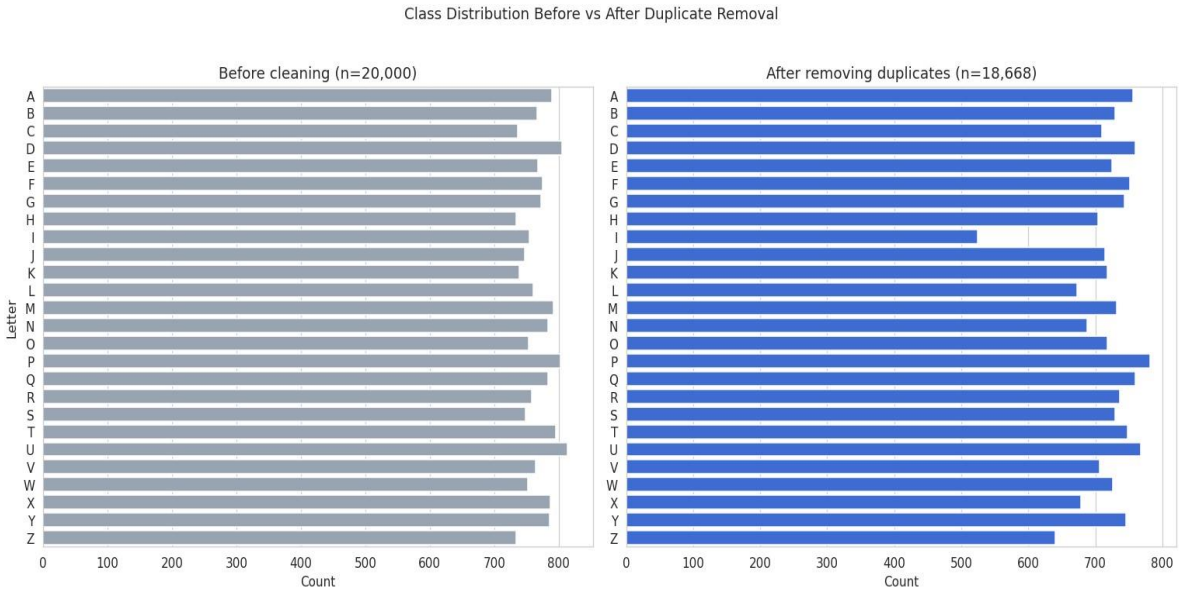


Figure 1. Class distribution before (n = 20,000) and after (n = 18,668) duplicate removal.

- **Duplicates are uneven across letters.** All 26 letters lose rows, but between 19 and 231 each; **I** loses the most (231), then X (109) and N (95). Class counts therefore move from 734–813 to 524–782, a mild imbalance (≈1.5:1) that needs no resampling but is the reason macro-averaged metrics are reported.
- **Outliers are concentrated in a few features.** The IQR rule flags xegvy most often (4,029 values on the raw data), followed by xy2br, x-bar and x2bar; y-box has none. Capping retains all rows (Figure 2).
- **The features are not heavily redundant.** The correlation heatmap (Figure 3) shows only a few moderately correlated pairs, yet PCA still reaches 95.2% of the variance with 13 of the 21 features, because the five engineered features are derived from the originals.
- **Edge and cross-moment features are the most discriminative.** In the ANOVA F-ranking on the final dataset edge_ratio leads (F = 1798.5), followed by x2ybr (1085.1) and y-bar (1038.9); positional features such as y-box and high rank lowest.

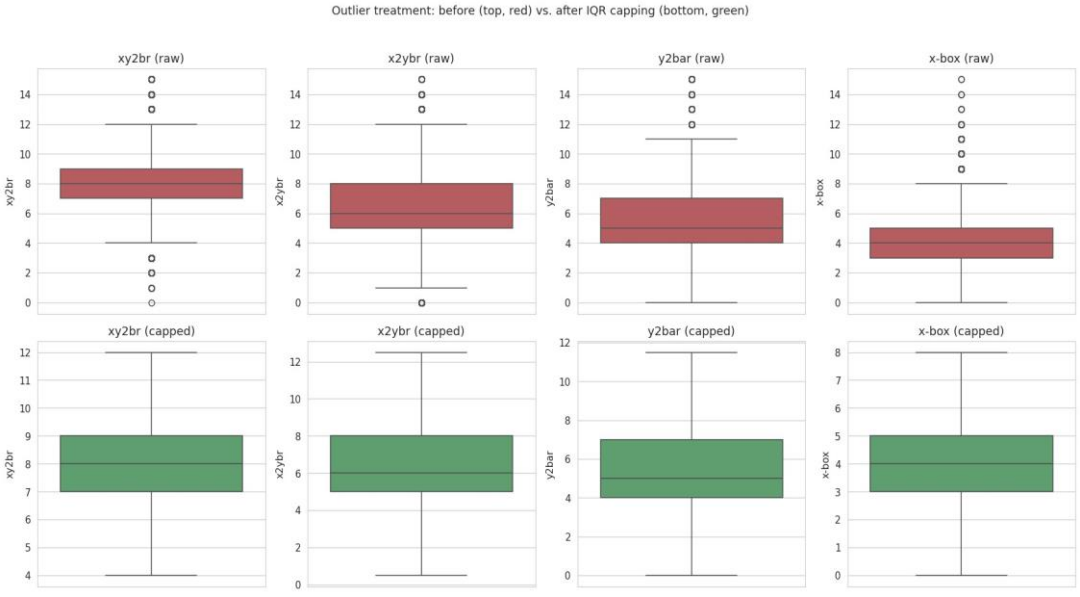


Figure 2. Feature distributions before and after IQR capping (Thrishodaran V.).

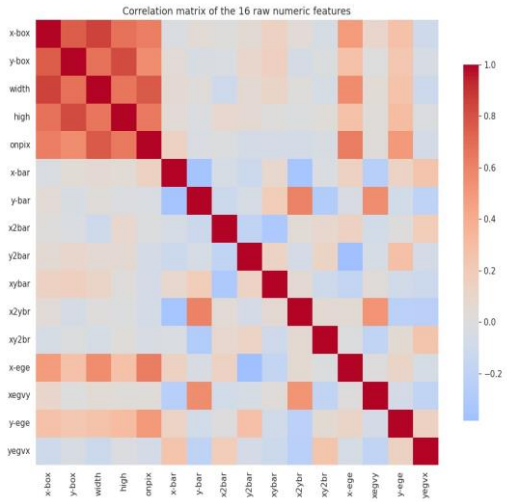


Figure 3a. Correlation heatmap of the features (Muhunthan R.).

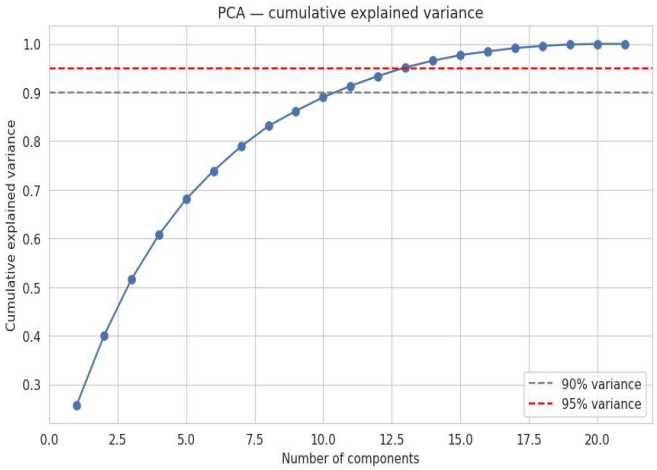


Figure 3b. Cumulative explained variance of the principal components (Danishiyan G.J.).

Model Design and Implementation

All models were implemented in Python (pandas, NumPy, scikit-learn, Matplotlib, Seaborn). Each member designed one model from a distinct category, trained an untuned baseline first, then tuned it, so that the benefit of tuning can be stated rather than assumed.

Member	Model (library)	Hyperparameters tuned	Tuning method	Models trained
Nirushiya A.	SVM – SVC (scikit-learn)	kernel, C, gamma, degree	GridSearchCV, 3-fold stratified CV	Baseline + 13 configs
Muhunthan R.	MLP – MLPClassifier	hidden layers, activation, alpha	GridSearchCV, 3-fold, early stopping	Baseline + 16 architectures
Thrishodaran V.	Decision Tree	criterion, max_depth, min_samples_split / leaf	GridSearchCV, 3-fold stratified CV	Baseline + shallow + 32 configs
Wijayagunasekara L.A.V.D.S.	Logistic Regression	penalty, solver, C	GridSearchCV, 3-fold stratified CV	Baseline + 6 configs
Danishiyan G.J.	PCA → KNN	n_components (2, 5, 11, 13, 21)	CV with PCA refitted per fold	5 settings + KNN baseline
Almeida K.K.O.	K-Means	number of clusters k	Elbow + silhouette sweep	k sweep; k = 20 and k = 26

Model suitability

- **SVM (RBF).** Many letters (for example I/J or R/K) occupy neighbouring regions of the feature space, so a flexible non-linear boundary is needed; the RBF kernel maps the data to a space where such boundaries become separable, and margin maximisation keeps it robust at ~15,000 training rows. The features are already on a 0–15 scale and were standardised, which benefits a distance-based kernel.
- **MLP.** A fixed-length vector of 21 numeric inputs suits a fully connected network, which learns curved boundaries through stacked non-linear layers rather than a kernel – a useful contrast with the SVM.
- **Decision Tree.** Axis-aligned threshold splits map naturally onto discrete integer descriptors, need no scaling, and give the only model whose decisions can be read as short rules (the interpretability counterpoint).
- **Logistic Regression.** Included as the linear baseline: it measures how much performance genuinely comes from non-linearity rather than tuning effort.
- **PCA → KNN.** The five engineered features are correlated with their parents by construction, exactly the redundancy PCA compresses. As PCA predicts nothing itself, a fixed KNN classifier measures how much predictive information survives.
- **K-Means.** Labels are withheld during fitting and used only for external validation (ARI, NMI), measuring how much class structure exists without supervision.

Common experimental protocol

All six models read the same processed CSV. The classifiers and the PCA study use one 80/20 stratified split with *random_state = 42* (14,934 training rows, 3,734 test rows). Tuning uses GridSearchCV with 3-fold stratified cross-validation on the training set only; the test set is used once for the final evaluation. K-Means is unsupervised and is fitted on all 18,668 rows with labels held out.

Worked example – SVM tuning (Nirushiya A.)

The grid covered three kernel families: linear (C = 0.1, 1, 10), RBF (C = 0.1, 1, 10 × gamma = scale, 0.01) and polynomial (C = 1, 10 × degree = 2, 3), giving 13 configurations and 39 cross-validation fits. The RBF kernel with C = 10 and gamma = scale won.

Kernel	C	gamma / degree	CV accuracy (mean ± sd)
RBF (selected)	10	scale	0.9547 ± 0.0030
Polynomial	10	degree 3	0.9369 ± 0.0029
RBF	1	scale	0.9148 ± 0.0029
Linear (best linear)	1	–	0.8434 ± 0.0007
RBF (worst)	0.1	0.01	0.6853 ± 0.0018

Table 4.1. Selected SVM configurations from the 13 tried (full ranking in the SVM notebook).

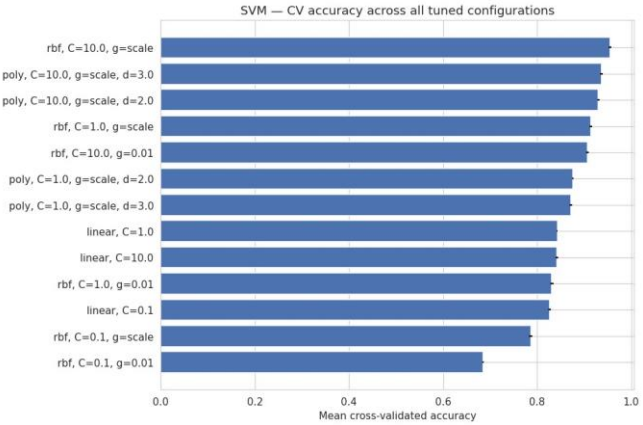


Figure 4. Cross-validated accuracy of all 13 SVM configurations.

Evaluation and Comparison

Metrics. Accuracy is meaningful because the 26 classes are near-uniform; macro-averaged precision, recall and F1 weight every letter equally and reveal any letter handled poorly. A confusion matrix shows *which* letters are confused. **Validation:** 3-fold stratified cross-validation during tuning (all models), a 5-fold cross-validation check for the Decision Tree (0.8518 ± 0.0066), and a final single evaluation on the untouched 20% test split. K-Means, which predicts no letter names, is evaluated with the Adjusted Rand Index (ARI) and Normalised Mutual Information (NMI) against the true letters.

Classification results (identical split, 3,734 test letters)

Rank	Model	Member	Accuracy	Precision	Recall	F1	Untuned baseline
1	SVM (RBF)	Nirushiya A.	96.46%	96.54%	96.45%	96.48%	92.96%
2	MLP	Muhunthan R.	94.67%	94.76%	94.57%	94.64%	91.64%
3	PCA (13) → KNN	Danishiyan G.J.	90.33%	90.64%	90.28%	90.37%	91.78% (KNN, no PCA)
4	Decision Tree	Thrishodaran V.	86.05%	86.15%	86.06%	86.07%	85.83%
5	Logistic Regression	Wijayagunasekara L.A.V.D.S.	77.32%	77.73%	77.20%	77.30%	76.89%

Table 5.1. Precision, recall and F1 are macro-averaged over the 26 letters.

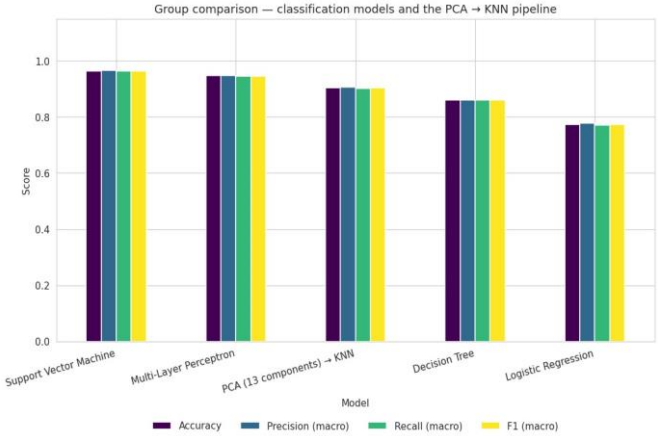


Figure 5a. All four metrics for each classifier.

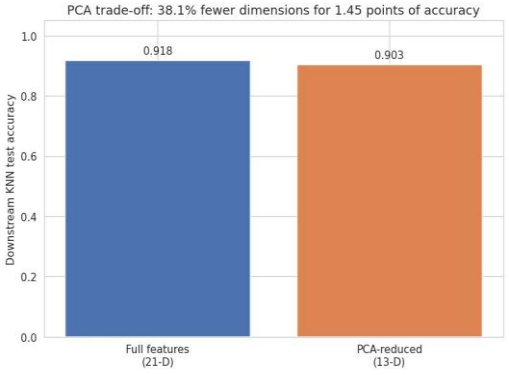


Figure 5b. PCA components vs accuracy and variance.

Clustering and dimensionality reduction

Model	Result
K-Means (k = 26)	ARI 0.1507, NMI 0.3638; silhouette-preferred k = 20 gives ARI 0.1434, NMI 0.3481 (silhouette 0.141)
PCA (21 → 13 components)	95.18% variance retained, 38.1% fewer dimensions; KNN accuracy 90.33% vs 91.78% on all 21 features (cost 1.45 points); 11 components already keep 91.4% (KNN 89.10%)

Comparison and conclusion

- **Best model: the RBF SVM** (96.46% accuracy, 0.9648 macro-F1; CV 0.9547). It improved 3.50 points over its untuned baseline, and the MLP is a close second (94.67%, +3.03 points). Both are flexible non-linear models, which suits letters separated by curved boundaries.
- **Evidence for non-linearity.** Logistic Regression, the only linear classifier among the final models, is 19.15 points behind the SVM. The SVM grid supports this from within one algorithm: its best linear kernel reaches 84.3% CV accuracy against 95.5% for RBF. The data are therefore strongly non-linear.
- **Tuning matters unevenly.** It gave large gains for the SVM and MLP but almost none for the Decision Tree (+0.22) and Logistic Regression (+0.43), whose accuracy is limited by model form, not by hyperparameters. A depth-5 tree reached only 38.24%, showing the underfitting extreme.
- **The Decision Tree** (86.05%, depth 20, 1,733 leaves) is the interpretable option, at a cost of about 10 points against the SVM.
- **Compression is cheap but not free.** PCA keeps 95% of the variance in 13 of 21 dimensions at a 1.45-point accuracy cost. **K-Means** finds real but weak structure (ARI ≈ 0.15), so supervision is essential for this task.
- **Fair comparison.** All models share the same data and split; precision, recall and F1 track accuracy closely for every model, which shows no large gap between accuracy and the macro metrics. Per-letter errors are examined next for the best model.

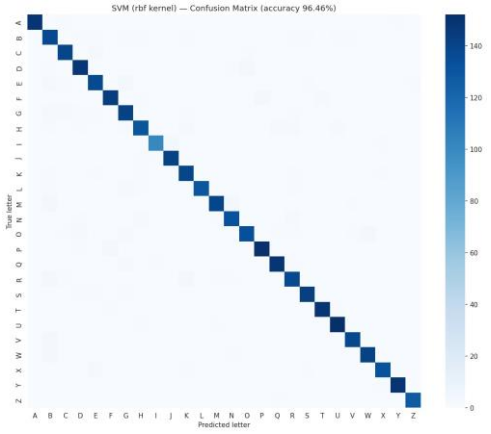


Figure 6a. SVM confusion matrix on the test set.

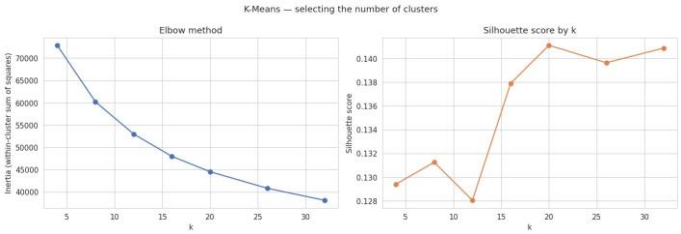


Figure 6b. K-Means elbow and silhouette sweep (Almeida K.K.O.).

The SVM misclassified 132 of 3,734 test letters. Its most frequent confusions are R→K, P→F and I→J (4 each), followed by W→B, V→B, R→B, O→W and M→N (3 each): visually similar glyphs, as expected. No single pair dominates, so the errors are spread thinly across letters rather than concentrated.

Limitations and improvements

- Final scores come from a single 20% test split; nested or repeated cross-validation would give a firmer ranking. Tuning used 3 folds and modest grids; a finer or randomised search around the winning SVM region (C = 10 was the largest value in the grid) could add a small gain.
- **Preprocessing was fitted before the split.** Outlier bounds and the StandardScaler were fitted on all 18,668 rows. Repeating the PCA → KNN experiment with both fitted on training rows only moved its accuracy from 90.33% to 90.36% (components and variance unchanged), so the effect is negligible there; the other models were not re-measured. Splitting first is the correct procedure.
- Capping and feature engineering can create new identical feature rows after de-duplication (68 such rows exist in the final file), a small residual leakage risk; de-duplicating again after Stage 3 would remove it.
- The MLP uses scikit-learn's MLPClassifier (no dropout or learning-rate schedules); SVM training cost grows roughly quadratically with sample count, and the model gives no interpretable rule for a prediction.
- No CNN was possible because only pre-extracted descriptors are available, not character images.

Ethical Considerations and Bias Mitigation

6.1 Cost of errors. OCR errors propagate: a misread letter in a name, address or identification code can misdirect mail, mismatch records or deny a service. At 96.46% accuracy about one letter in 28 is wrong, so any deployment handling consequential text needs human review of low-confidence or high-stakes fields, and should never be treated as error-free.

6.2 Dataset bias and generalisability. The data contain only capital English letters rendered in 20 commercial fonts, not natural handwriting, lower-case letters, digits or non-Latin scripts such as Sinhala and Tamil. The reported accuracy therefore cannot be assumed for real scanned documents, handwriting or other languages, and would disadvantage writers and scripts that are absent from the training data. Any real system would need representative data from its actual users and scripts before use.

6.3 Data-quality bias. 6.66% of the rows were exact duplicates, unevenly distributed (letter I lost 231 rows). Leaving them in would have inflated test accuracy by letting copies of a training row appear in the test set. De-duplication slightly altered the class balance (524–782 per letter), which was reported openly and handled with macro-averaged metrics rather than ignored.

6.4 Privacy and transparency. The dataset is public research data with no personal information. Real OCR pipelines process personal documents, so deployment would require consent, access control, secure storage and compliance with data-protection law, including Sri Lanka's Personal Data Protection Act No. 9 of 2022. Because the SVM and MLP are hard to interpret, the group recommends keeping the Decision Tree or confusion-matrix analysis available for auditing which letters fail.

6.5 Mitigation steps taken.

- Stratified splits and stratified cross-validation folds preserve the class mix in every subset.
- Macro-averaged precision, recall and F1 and a confusion matrix are reported, so weak letters are visible.
- Exact duplicates were removed to avoid leakage; a ratio-feature defect and the pre-split fitting limitation were found, documented and measured rather than hidden.
- Honest scope statements: results apply to machine-printed capitals from these fonts only.

AI Tool Usage Declaration and Transparency

The group used generative AI tools as a support tool only. The core work was done by the group members: understanding the assigned dataset, implementing and explaining the six preprocessing techniques, building, tuning and running the six model notebooks, and producing every number and figure in this report by executing the code.

Tool (version)	Aspect supported	Extent	How outputs were verified or owned
Claude (Anthropic); model version as provided in the Claude chat app	Reviewing the project notebooks and results against the raw data; spotting inaccurate statements (class-balance claim, confusion-pair claim, overstated wording)	Moderate	Counts, accuracies and metrics were recomputed from the raw data and the saved JSON files and matched the notebooks; corrections were applied by the group to the notebooks and report.
Claude (Anthropic)	Structuring and drafting this report from the group's executed results (Sections 1–9), including draft wording of the contribution notes in Section 8	Extensive (drafting); none for results	Every figure and number was copied from notebook outputs and result files; members read and take responsibility for the sections covering their own work.
Claude (Anthropic)	Explaining the pipeline and viva-preparation questions	Minimal	Used for understanding only; members can explain and run their own notebooks.

Ownership. The group takes responsibility for the accuracy of all submitted work. Each member owns one preprocessing technique and one model and is expected to run and explain that notebook without AI assistance in the viva.

Reflections and Lessons Learned

Group lessons

- **Code that runs is not code that is correct.** The ratio-feature defect produced no error and plausible summary statistics, and was found only by inspecting the distribution of the engineered features; every downstream result had to be regenerated.
- **Validation order matters.** Fitting the capping bounds and scaler before the split is a form of leakage. Measuring its effect (90.33% → 90.36% for PCA → KNN) turned a worry into a quantified, documented limitation.
- **Check claims against the data.** A review showed that duplicates were not spread evenly across letters and that some confusion pairs named in the text were not the ones the matrix shows; claims were corrected to match the evidence.
- **Compare like with like.** Accuracy is undefined for clustering and PCA, so each model is reported on metrics suited to its task rather than forced into one league table.
- **What we would do differently:** agree a leak-free split-then-fit helper before anyone tunes, compare preprocessing variants (not only hyperparameters) for every model, and use repeated cross-validation instead of one test split.

Individual contributions and learning

Member	Contribution and key learning
Nirushiya A.	Removed 1,332 duplicates (letter I most affected) and tuned the SVM over 13 configurations (96.46%). Learned that cleaning steps change class balance and must be reported honestly, and that the kernel choice (linear 84.3% vs RBF 95.5% CV) matters far more than small changes to C.
Thrishodaran V.	Capped outliers with the IQR rule without deleting rows and built the Decision Tree (86.05%). Learned that capping preserves class balance, and that an unrestricted tree overfits while a depth-5 tree underfits (38.24%).
Wijayagunasekara L.A.V.D.S.	Encoded the 26-class target and built the Logistic Regression baseline (77.32%). Learned that near-flat accuracy across the grid signals a model-form limit, not a tuning problem.
Almeida K.K.O.	Compared StandardScaler and MinMaxScaler and ran the K-Means study (ARI 0.151, NMI 0.364). Learned that the silhouette-preferred k (20) differs from the true class count (26), so cluster metrics need an external check.
Danishiyan G.J.	Engineered five features and studied PCA → KNN (13 components, 95.18% variance, 90.33%). Learned that a zero-denominator guard must be checked by inspecting values, and that PCA trades 1.45 points of accuracy for 38.1% fewer dimensions.
Muhunthan R.	Ranked features (ANOVA F-test, Random Forest importance) and tuned 16 MLP architectures (94.67%). Learned that tuning lifts a neural network by about three points and that it still trails a well-tuned kernel SVM on this tabular data.

References

[1] Slate, D.J. (1991) *Letter Recognition Data Set*. UCI Machine Learning Repository, University of California, Irvine. https://archive.ics.uci.edu/dataset/59/letter+recognition

[2] Frey, P.W. and Slate, D.J. (1991) 'Letter recognition using Holland-style adaptive classifiers', *Machine Learning*, 6(2), pp. 161–182.

[3] Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.

[4] Cortes, C. and Vapnik, V. (1995) 'Support-vector networks', *Machine Learning*, 20(3), pp. 273–297.

[5] Kaufman, S., Rosset, S., Perlich, C. and Stitelman, O. (2012) 'Leakage in data mining: Formulation,detection, and avoidance', *ACM Transactions on Knowledge Discovery from Data*, 6(4), pp. 1–21.

[6] Harris, C.R. et al. (2020) 'Array programming with NumPy', *Nature*, 585, pp. 357–362.

[7] McKinney, W. (2010) 'Data structures for statistical computing in Python', *Proceedings of the 9th Python in Science Conference*, pp. 56–61.

[8] Hunter, J.D. (2007) 'Matplotlib: A 2D graphics environment', *Computing in Science & Engineering*, 9(3), pp. 90–95.

[9] Anthropic. *Claude* (AI assistant), used as declared in Section 7. https://www.anthropic.com